SPHERE WEB AUTOMATION

Course TOC Script Step-by-Step Walkthrough

An element-by-element map of how `tests/CourseTOC.spec.ts` verifies the course overview page — the page's unusually slow banner, the button that changes element type as you learn, and why this is the most strongly-asserted suite in the set.

Script tests/CourseTOC.spec.ts · 525 lines · 21 tests in 5 groups

Target /app/toc/<courseId>/overview (login + COURSE_NAME required)

Run npm run run:course-toc

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, entry points
2	Page anatomy	Diagram — five groups on one page
3	The readiness model	Diagram — why tabs are not "ready" · the four timeout knobs
4	Helpers and harness	Seven helpers, four auth modes
5	Group 1 — Header	5 tests · lines 273–306
6	Group 2 — Action icons	6 tests · lines 308–384
7	Group 3 — Resume/Start and tabs	4 tests · lines 386–431
8	Group 4 — Description and licence	3 tests · lines 433–463
9	Group 5 — Navigation	3 tests · lines 465–524
10	What green actually proves	Diagram — 15 assert · 3 conditional · 3 observe
11	Artifacts and run variants	Reports, console output, grep recipes
12	Failure modes and fixes	Troubleshooting reference

How to read this document

Sections 5 through 9 walk the script in file order, one table per group, following the `// ── GROUP ──` banners in the source. Each row gives the **action**, the exact **locator**, and what the test does with the result — marked by strength:

Marker	Meaning
assert	An `expect()` always runs. A missing or wrong element fails the test.
conditional	The test calls `skipIfAbsent()` first. If the control is not on this course the test *skips*; if it is, a real `expect()` runs.
observe	No `expect()` at all. The test performs the action and prints what happened for a human to read.

The ratio is what sets this suite apart from the others in the set: **15 of 21 tests assert unconditionally**, 3 more assert behind a skip guard, and only 3 merely observe. Section 10 tallies it group by group and explains why the balance is so much healthier here than on, say, My Courses.

> **Line numbers in this document are live**
> Every line reference points at `tests/CourseTOC.spec.ts` as it stands today — 525 lines, including the per-line explanatory comments that were added across the suite. They are safe to click through to.

1 What this script does

It is the sweep of the course overview page — the screen a learner lands on after searching for and opening a course, and the screen they come back to between every module.

The URL shape is `/app/toc/<courseId>/overview`, and the page carries a lot: the course title and its metadata badges, three action icons, the Start/Resume banner, three tabs, a description, and a licence block. This spec puts **one granular test on each of those elements** — 21 tests in five groups — so a failure names the exact control that broke rather than "the course page".

> **Its scope was carved out of the course-completion suite**
> `CourseComplete.spec.ts` drives a whole course end to end and deliberately *does not* click around the overview page, because doing so disturbs the Resume state it depends on. Its comment at line 2211 says as much: *"the full overview element sweep lives in CourseTOC.spec.ts now."* This file is that sweep, and the separation is why both can be trusted.

Two inputs, both required

Input	Why it is needed
`a logged-in session`	The TOC is behind auth. The suite imports `test` from `./helpers/authedTest` (line 21), so every test arrives with a logged-in context already established.
`COURSE_NAME`	Line 28. There is no default. Without it, `beforeEach` calls `test.skip()` on line 223 and all 21 tests skip — they do not fail, and they do not run against a blank page.

That second point matters when reading a report: **21 skipped is the signature of a missing `COURSE_NAME`**, not of a broken page. The skip message says so explicitly — *"Set COURSE_NAME to run Course TOC coverage"*.

How a test gets to the page

Every test starts from the home page and searches. There is no hard-coded course URL anywhere in the file, because course ids differ per environment. `searchAndOpenCourse()` (line 47) types the name into the search box, presses Enter, and clicks the first result whose title starts with the same 8 characters — a prefix match, because result cards truncate long titles.

Once one test has opened the course, the URL is cached in `tocUrl` (line 197) and every later test takes a much cheaper path: navigate straight there, verify it landed, done. Section 3 diagrams both paths and the verification between them.

The four things it covers

Header metadata

- Course title
- CNE hours badge
- Duration, in any of four formats
- Rating and review count
- The `By:` author line

Interactive controls

- Details / Certificate / Share icons
- Each popup opens *and closes*
- Start / Resume banner button
- Overview / Modules / Discussion tabs
- The module list actually has modules

Content and exits

- Description section
- Licence terms, Creative Commons link
- Author link to the organisation page
- Back icon leaves the overview
- Start/Resume opens the player

Entry points

Command	What it runs
`npm run run:course-toc`	All 21 tests, headed, against staging. The full walkthrough this document describes. Set `COURSE_NAME` first.
`npm run report:app`	This spec alongside Account, My Courses and Competency — the authenticated-app blob report.
`… --grep "Certificate"`	Just the two certificate tests. Every test title starts with `TOC:`, so grepping on a control name is unambiguous.

```
# PowerShell — the one-liner from the file header (line 16)
$env:COURSE_NAME="Implementing Respectful Maternity Care In Healthcare Facilities"
npm run run:course-toc
```

Three minutes per test

Line 194 sets `test.setTimeout(3 * 60 * 1000)`. That is not defensive padding — the banner alone is allowed 90 seconds (Section 3), and a slow-path re-open makes up to three trips through search before giving up. A 30-second default would fail this suite on a healthy site.

2 Page anatomy

Everything this suite touches lives on one screen. The five groups in the source map cleanly onto five bands of that screen, top to bottom.

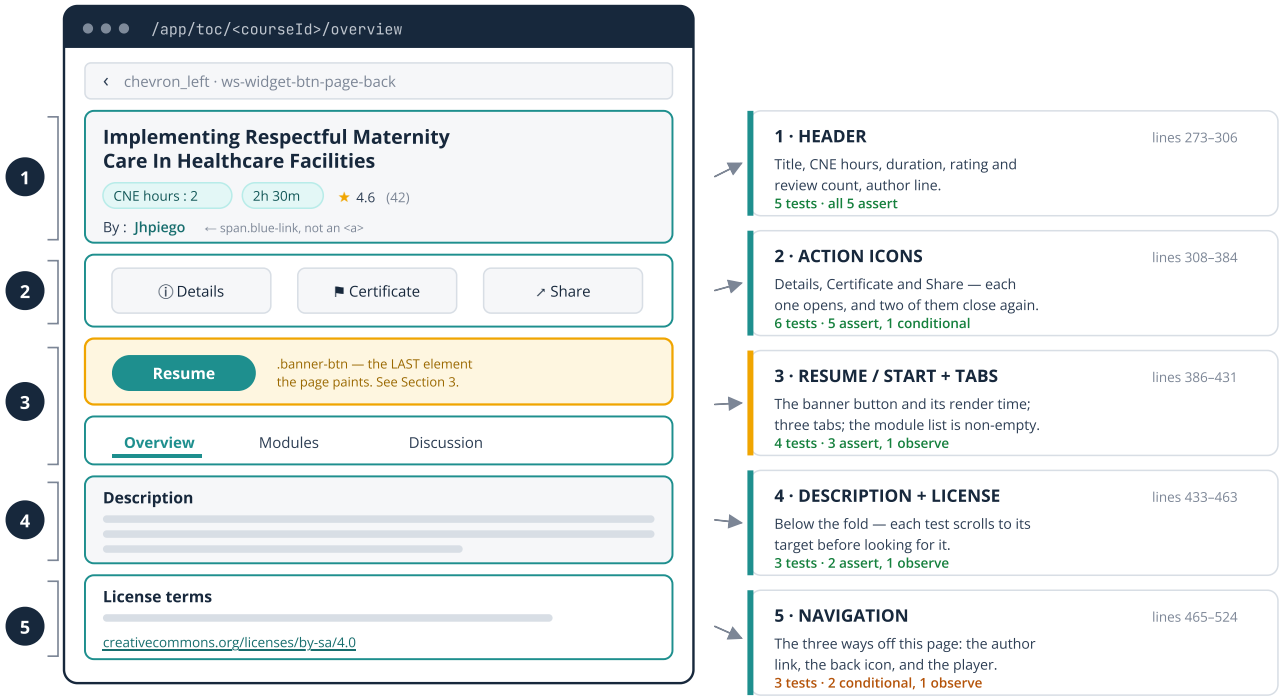


Figure 1 — The course TOC overview, with the five source groups mapped onto the five bands of the page. The gold band is the one the whole readiness model is built around.

> **Why the groups run top to bottom**
> It is not a coincidence. The file is ordered the way a person reads the page, so when a report shows five consecutive failures you can read them as "everything below the banner is missing" rather than having to reconstruct which control lives where.

The two markup details the suite had to learn

Element	What is surprising about it
Action icon `line 178`	An icon is a `div.toc-icon-action` wrapping an icon and a label span. **The `(click)` handler is on the div** — clicking the inner label does nothing at all. `actionIcon()` therefore locates the wrapper and filters it by the label text, rather than clicking the text directly.
Author link `line 468`	The organisation name after `By:` is a `span.blue-link`, **not an anchor**. `getByRole('link')` finds nothing. The test uses the structural class, with a text filter naming common org types as a fallback.

Both are the kind of thing that only shows up once you have watched a test click something and seen nothing happen. Both are documented in the source at the helper that solves them, so the next person does not rediscover them.

3 The readiness model

This is the idea the whole file is built on: **the tabs appearing does not mean the page is ready.** Everything else — the 90-second timeout, the two-stage gate, the fast and slow paths — follows from that one measurement.

The TOC overview paints in layers. The shell and the tabs come from the course metadata and are on screen in well under a second. The Start/Resume banner is different: it waits on the enrolment and progress calls, so it is the *last* thing to appear. On a clean load that is 1.6 to 2.8 seconds. Coming back from the player — which is exactly what a learner does between modules — it has been measured at around 70 seconds.

> **The bug this replaced**
> The original tests used a plain `expect(banner).toBeVisible()`, which carries Playwright's default 10-second window. On a clean load it passed. After a round trip through the course player it failed every time — **while the page was still painting correctly**. The suite was reporting a product failure that was really a test failure.

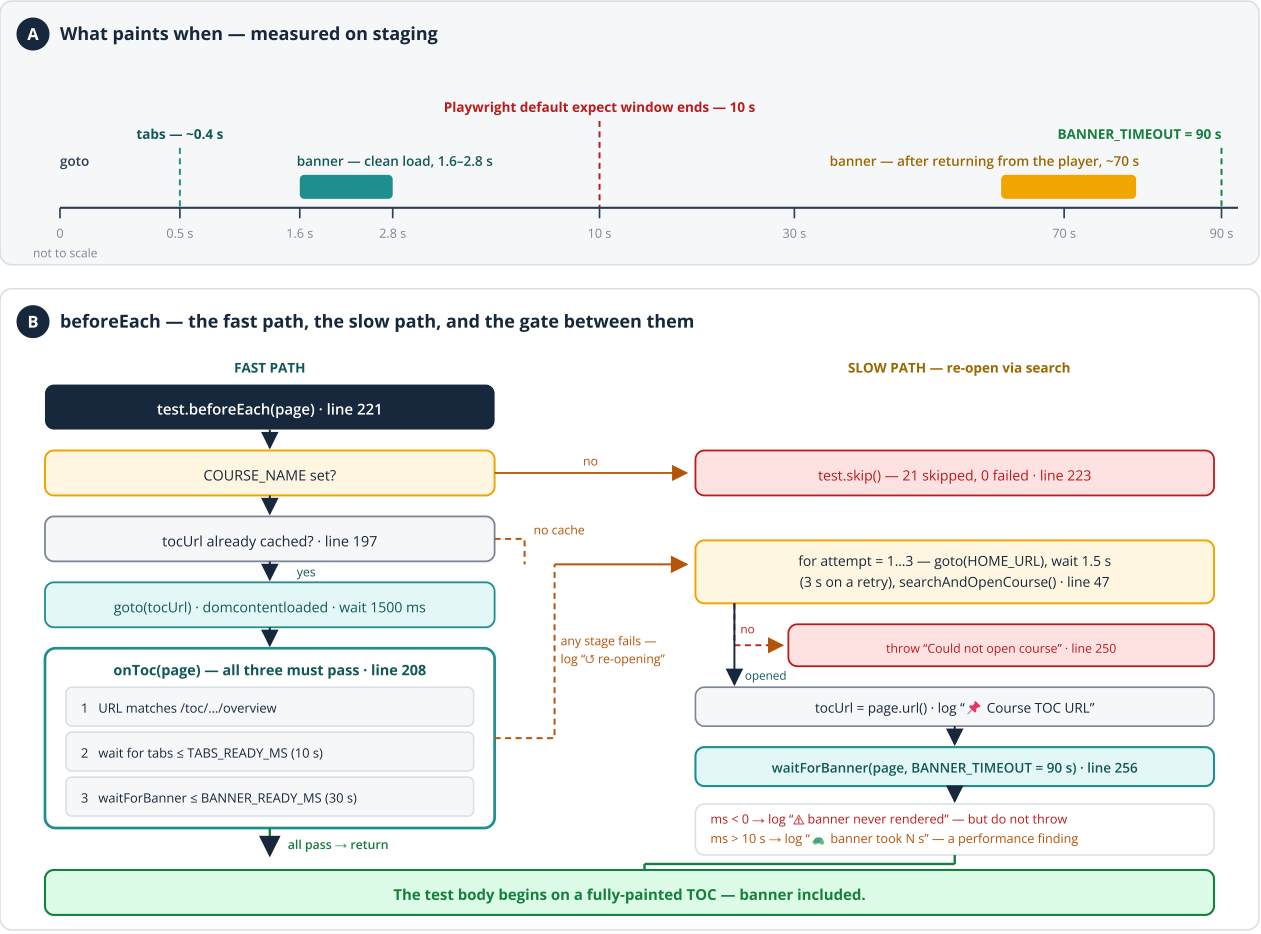


Figure 2 — (A) the paint order that forced the design, and (B) the `beforeEach` that guarantees every one of the 21 tests starts from the same fully-rendered state.

3 The readiness model (continued)

Why `isVisible()` could not be used for the gate

The comment at lines 204–207 records a real regression. `onToc()` originally used `isVisible()` to check for the tabs. That call **does not wait** — its `timeout` option is ignored, and it answers about the current instant only. Used milliseconds after a `goto`, it reported "not the TOC" on a page that was about to render perfectly, and sent *every* test down the slow re-open-via-search path. The suite still passed; it just took several times longer than it needed to.

The fix is the `.then(true).catch(false)` idiom on line 213 — it turns a `waitFor()` into a boolean without letting the rejection escape:

```
// line 212-213 — a wait that answers yes/no instead of throwing
const tabs = await page.getByText(/^(Details|Overview|Modules)$/i).first()
  .waitFor({ state: 'visible', timeout: TABS_READY_MS }).then(() ⇒ true).catch(() ⇒ false);
```

The three timeouts are three different questions

It would be simpler to have one banner timeout. There are three because they answer different questions, and collapsing them would make the suite either slow or flaky.

Constant	Default	The question it answers
`BANNER_TIMEOUT` line 34	**90 s**	"How long may the banner take before we call it broken?" Used after a fresh open, where there is no fallback left — so it has to cover the ~70-second worst case with headroom.
`BANNER_READY_MS` line 37	**30 s**	"Is the cached URL good?" Deliberately shorter, because a failure here is cheap — it just falls through to the slow path. Waiting the full 90 seconds to discover the cache is stale would be wasted time.
`TABS_READY_MS` line 39	**10 s**	"Is this even a rendered TOC?" The tabs paint in under half a second, so 10 seconds is already twenty times the observed value — it exists to fail fast on a wrong page, not to wait for a slow one.
`SLOW_BANNER_MS` line 41	**10 s**	Not a timeout at all. "Was this slow enough that a human should know?" Crossing it prints a 🚗 line and nothing else. Section 11 explains how to mine those lines out of a run.

All four read from the environment, so a slow CI box can be given more room without editing the file:

```
# give the banner three minutes and raise the slowness bar to 20 s
$env:TOC_BANNER_TIMEOUT="180000"; $env:TOC_SLOW_BANNER_MS="20000"; npm run run:course-toc
```

The banner button changes element type as you learn

This is the other discovery in the file, and it is documented at length in the doc-comment above `startOrResumeBtn()` (lines 86–100). The same visual control is emitted as two different elements depending on progress:

Learner state	What the DOM actually contains
Not started, or in progress	`<button class="banner-btn">Start | Resume</button>`
100% complete	`<a class="banner-btn resumeButton">Start Again</a>`

> **Verified on staging: 0 matches as a link, 1 via `.banner-btn`**
> On a completed course `getByRole('button')` finds nothing, because the control is no longer a button — and **the link role does not rescue it either**: the measurement recorded in the source is *zero* matches as a

link against one match via the class. Neither role is dependable, so `startOrResumeBtn()` puts `.banner-btn` first and keeps both roles behind it as belt-and-braces. The failure this prevents is the most awkward kind: it appears only on accounts that have finished something.

```
// line 101-107 — class first, roles as a fallback, .first() to stay strict-mode safe
function startOrResumeBtn(page: Page): Locator {
  return page.locator('.banner-btn, [class*="resumeButton"]')
    .or(page.getByRole('button', { name: /^(resume|start|start again)$/i }))
    .or(page.getByRole('link',   { name: /^(resume|start|start again)$/i }))
    .first();
}
```

4 Helpers and harness

Seven helpers sit between the tests and the page. They exist so that a test body can read as one sentence about one control, and so that a markup change is fixed in one place rather than twenty.

Helper	Line	What it does, and the problem it solves
`searchAndOpenCourse`	47	Types `COURSE_NAME` into the home search box, presses Enter, and clicks the first result. Two details matter: it matches on the **first 8 characters only**, because result cards truncate long titles; and it treats a **changed URL as the proof** that the click opened something, rather than trusting the click.
`startOrResumeBtn`	101	Returns a locator for the banner control across both of its element forms. Covered in Section 3.
`waitForBanner`	110	Waits for that control and **returns how long it took in milliseconds**, or `-1` if it never appeared. Returning a number rather than a boolean is what lets the caller distinguish "fine", "slow enough to report", and "broken" — see the 🐢 line in Section 11.
`startOrResume`	125	Actually clicks the control, trying **six locators in order** and stopping at the first visible one. Returns whether the URL changed. Its last resort (line 151) is a URL test for `/viewer/`, `/play/` or `/learn/` — if we were already inside the player, that counts as success.
`closeAnyPopup`	157	Best-effort dismissal. Tries **five shapes of close button** in one selector, falls back to the Escape key, then waits 400 ms for the dialog animation. Its timeouts are deliberately tiny (700 ms to look, 1500 ms to click) because this is cleanup, not a check.
`actionIcon`	178	Finds a `div.toc-icon-action` and narrows it by label text. Exists because the click handler is on the wrapper div, not the label span inside it.
`skipIfAbsent`	265	Logs ⏭ and calls `test.skip()` when an optional control is missing. Declared *inside* the describe block because `test.skip(true, …)` only works from within a test's scope.

Alongside them is one constant, `POPUP` (line 185), which lists six container shapes in a single selector string:

```
'[role="dialog"], .cdk-overlay-pane, .mat-mdc-dialog-panel, [class*="modal"], [class*="popup"],
[class*="dialog"]'
```

Angular Material has changed its dialog class names across versions, and the app mixes hand-rolled overlays with Material ones. Listing all six means a rename in any single shape does not break the four popup tests at once.

> **Deliberately self-contained**
> The banner above the helpers (line 44) says "self-contained so this file is independent of `CourseComplete`". `searchAndOpenCourse`, `startOrResume` and `closeAnyPopup` all have cousins in the completion suite. Duplicating them was the choice: the completion suite's versions are tuned for driving a course to the end, and coupling the two files would mean a change made for one silently altering the other.

How the suite gets a logged-in page

Line 21 imports `test` and `expect` from `./helpers/authedTest` rather than from Playwright directly. That harness picks an auth mode from the environment, first match winning:

#	Trigger	How it authenticates
1	`USE_CDP=1`	Attaches to a Chrome you are already logged into, over the DevTools protocol. The local-development mode — on localhost, cookie injection returns 419, so this is the only thing that works. No `storageState` file is written, and the harness never closes a browser you own.
2	`USE_FRESH_USER=1`	Reuses the session of an account minted by the create-account suite, from `.auth/fresh-user.json`. Falls back to credential login with a warning if that file is missing.
3	a saved state file	Loads a previously captured `storageState`.
4	`TEST_USER_EMAIL` + `TEST_USER_PASSWORD`	Drives a real UI login once, saves the state, and hands every test a context built from it. This is what CI uses against staging and production.

If none of the four is configured the harness **throws immediately** with a message listing all of them — it fails in setup rather than running 21 tests against a login screen.

Not serial, on purpose

Line 191 carries a one-line note that is easy to skip past: *"NOT serial — each test resets to the TOC in `beforeEach`, so one failure must not skip the rest."*

That is a real trade. `test.describe.serial` would let the tests share a single page load and run much faster. But under serial, the first failure aborts every test after it — so a broken Details popup would hide the fact that the licence block is also missing. Here, each test pays the cost of returning to a known state and, in exchange, **a report of 21 results is 21 independent facts**.

The tests do still share one thing: `tocUrl` (line 197), the cached course URL. It is a performance cache, not shared state — every test re-verifies it through `onToc()` before trusting it, and silently re-opens the course if it has gone stale.

> **One test self-heals mid-body**
> Line 388, in the Resume/Start test, re-checks `onToc(page)` and navigates back if needed. That is there because in CDP mode the browser belongs to a human — a stray click in the window the tests are driving can move the tab off the TOC between `beforeEach` and the assertion.

5 Group 1 — Header

Five tests, lines 273–306. One per piece of metadata in the block at the top of the page. All five assert unconditionally.

#	Test · line	Locator	What it proves
1	**Course title is visible** line 273	`getByRole('heading', { name: RegExp(COURSE_NAME.slice(0,18)) }) .or(getByText(same)) .first()`	assert That the course we searched for is the course we are looking at. It matches the **first 18 characters** — long enough to be unique, short enough to survive title truncation — and accepts either a real heading or plain text, because the title is not always marked up as a heading. Logs the first 60 characters it found.
2	**"CNE hours" badge** line 282	`getByText(/CNE hours?\s*:/i).first()`	assert The continuing-education credit badge rendered. **The colon is doing real work** — without it the regex would also match a sentence in the description that happens to mention CNE hours, and the test would pass on a page with no badge at all.
3	**Course duration** line 288	`getByText(/\b\d+\s*h(\s*\d+\s*m)?\b |\b\d+\s*min |\b\d+\s*hours?/i).first()`	assert A duration is displayed in any of the four shapes the app emits: `2h`, `2h 30m`, `45 min`, `3 hours`. One alternation rather than four tests, because which format appears is a content decision, not a bug.
4	**Rating and review count** line 294	`getByText(/\(\s*\d+\s*\)/).first()`	assert A parenthesised review count such as `(42)` is on the page, and the value is logged. See the note below on what this does *not* check.
5	**Author "By: <org>"** line 301	`getByText(/^\s*By\s*:/i).first()`	assert The attribution line rendered. **Anchored at the start** with `^`, so it cannot be satisfied by the words "by:" appearing in the middle of a description paragraph.

> **Test 4 is named for more than it checks**
> The title says "rating and review count"; the assertion covers only the review count. `/\(\s*\d+\s*\)/` matches the bracketed number, and nothing looks at the stars — a page rendering `(42)` with the rating missing would pass. Fix it with a second assertion on the numeric rating, or rename the test to "review count is visible".

That is the only over-promising title in this file. It is worth calling out precisely because there is just one: on the My Courses suite the equivalent list ran to eight tests.

Anchored versus unanchored

Three of the five regexes above are anchored; two are not. The rule — which holds across every group that follows — is that an anchor goes on text short and common enough to appear inside *other* text (`By:`, `Details`, `Overview`), and is left off where the pattern is already specific, as with the duration formats.

6 Group 2 — Action icons

Six tests, lines 308–384. The three icons under the header, and what each does when clicked. The only group that tests a control *closing* as well as opening.

#	Test · line	Locator	What it proves

6	**All three icons visible**
	line 308
		`getByText(/^Details$/i).first()`
		`getByText(/^Certificate$/i).first()`
		`getByText(/^Share$/i).first()`
			assert Three separate `expect()` calls in one test. All anchored, so `Certificate` matches the icon label and not the phrase "certificate of completion" elsewhere on the page.

7	**"Details" opens its popup** line 316	`actionIcon(page, /^details$/i).click()` `expect(locator(POPUP).first()).toBeVisible({timeout:5000})`	assert The click reaches a handler and a dialog opens within 5 s. Uses `actionIcon()`, so it clicks the wrapper div — clicking the label span would do nothing and the test would fail on a working page.

8	**"X" closes the details popup**
	line 326
		`open → expect(popup).toBeVisible()`
		`closeAnyPopup(page)`
		`expect(popup).not.toBeVisible({timeout:4000})`
			assert Two assertions with a reason. The first confirms the popup opened, **so a failure on the second can only be about closing** — without it, a dialog that never opened would report as "close is broken". A modal that cannot be dismissed traps the learner, which is why this is asserted rather than observed.

9	**"Certificate" shows a popup**
	line 339
		`locator(POPUP)`
		`.or(locator('canvas'))`
		`.or(getByText(/complete the course|certificate|finish the course|not.*(eligible|complete)/i))`
		`.first().isVisible({timeout:6000})`
		`expect(shown).toBe(true)`
			assert That the control *responds*. This icon legitimately does two things: on a completed course it opens the certificate dialog, drawn onto a `<canvas>`; in progress it shows a "complete the course first" message. Both are correct, so both count — **silence is the only failure**.

10	**"Close" closes the certificate popup**
	line 355
		`skipIfAbsent(popup visible, 'Certificate popup')`
		`getByRole('button', { name: /^close$/i })`
		`… else closeAnyPopup(page)`
		`expect(popup).not.toBeVisible({timeout:4000})`
			conditional If nothing opened it **skips rather than fails** — test 9 already covers that case, and failing twice for one defect makes a report harder to read. When something did open, the close assertion is as hard as test 8's. Prefers a labelled Close button, falling back to the generic dismissal.

11	**"Share" shows options**
	line 372
		`locator('[class*="share"], [class*="social"]')`
		`.or(getByRole('link', { name: /facebook|twitter|whatsapp|linkedin|copy|telegram/i }))`
		`.or(getByRole('button', { name: /…|copy link/i }))`
		`.first() → toBeVisible({timeout:5000})`
			assert Three independent signals, any one of which means the panel opened: a share- or social-styled container, a social *link*, or a social *button*. The app has shipped this panel as both links and buttons, so demanding one shape would break on a refactor rather than on a regression.

Every popup test cleans up after itself

Tests 7, 9 and 11 all end with `closeAnyPopup(page)`, even though `beforeEach` re-navigates anyway. It is cheap insurance: a dialog left open can swallow the next test's clicks in the moment before the navigation lands, and that failure looks like a broken control rather than leftover state.

7 Group 3 — Resume/Start and tabs

Four tests, lines 386–431. The banner button the whole readiness model was built for, and the three tabs beneath it.

#	Test · line	Locator	What it proves

12	**Resume/Start is visible**
	line 386
		`if (!await onToc(page)) goto(tocUrl)`
		`const ms = await waitForBanner(page, BANNER_TIMEOUT)`
		`expect(startOrResumeBtn(page)).toBeVisible({ timeout: 5_000 })`
			assert The banner rendered, in whichever element form this learner's progress produces. It also **reports the render time** and prints a 🐢 line above `SLOW_BANNER_MS`. Note the 5-second expect window: `waitForBanner` has already done the waiting, so a long window here would only delay a genuine failure.

13	**Three tabs are visible**
	line 401
		`getByText(/^Overview$/i).first()`
		`getByText(/^Modules$/i).first()`
		`getByText(/^Discussion$/i).first()`
			assert All three tab labels rendered. Anchored, so `Overview` matches the tab and not the word appearing in the description.

14	**"Modules" opens the module list**
	line 409
		`click Modules · wait 1500 ms`
		`locator('mat-list-item, ws-web-course-toc-card, [class*="toc-card"], [class*="module-item"], mat-tree-node').count()`
		`expect(count).toBeGreaterThan(0)`
			assert The one counting assertion in the file, and the most valuable test in the group: **an empty module list means the course content failed to load**, which a visibility check on the tab label would never catch. Five selector shapes are tried because the app has shipped this list as three different components.

15	**"Discussion" is clickable** line 424	`click Discussion · wait 1200 ms` `console.log(page.url())`	observe No `expect()`. The click either works or throws on the click itself; where it lands is **logged, not asserted**, because discussion is an in-page tab on some builds and a separate route on others. Asserting either shape would make the test fail on a deployment difference.

> **Test 15 is the weakest test in the file — and that is a defensible choice**
> It will fail only if the Discussion label is absent or unclickable; a tab that opens an empty or broken panel passes. Strengthening it means deciding what "Discussion works" should mean across both builds — a real product question, not a test one. Until that is settled, printing the destination URL is honest: the log line tells a reviewer exactly where the click went, which is enough to spot a change by eye.

Where the count in test 14 comes from

The five selectors are not alternatives tried in order — they are one locator matching any of them, and `.count()` returns the total across all five. On a build that renders the list as `ws-web-course-toc-card`, the other four simply contribute nothing. The number is printed either way:

```
✅ Modules tab — 14 module/resource item(s)
```

so an unexpected drop stays visible across runs even while the assertion — greater than zero — remains satisfied.

8 Group 4 — Description and licence

Three tests, lines 433–463. The two content blocks at the bottom of the overview, plus the licence link inside the second one.

#	Test · line	Locator	What it proves

16	**"Description" section** line 433	`click Overview tab (errors swallowed) · wait 600 ms`
		`const desc = getByText(/^Description$/i).first()`
		`desc.scrollIntoViewIfNeeded()`
		`expect(desc).toBeVisible()`
			assert The description heading is on the page. It **clicks back to Overview first**, because test 14 or 15 may have left the page on another tab — and it swallows any error from that click, since being on Overview already is not a failure.

17	**"License terms" section** line 445	`const license = getByText(/license\s*terms?/i).first()`
		`license.scrollIntoViewIfNeeded()`
		`expect(license).toBeVisible()`
			assert The licence block rendered. The trailing `s?` accepts both "License term" and "License terms" — the app has used both, and which one appears is a copy decision rather than a defect.

18	**Creative Commons link** line 454	`getByRole('link', { name: /creativecommons|creative commons/i })`
		`.or(locator('a[href*="creativecommons.org"]'))`
		`.first().scrollIntoViewIfNeeded()`
		`skipIfAbsent(…, 'Creative Commons link')`
		`console.log(await ccLink.getAttribute('href'))`
			observe Genuinely optional — not every course is CC-licensed — so a missing link **skips**. When the link is there the test prints its `href` and stops; there is no `expect()` on the URL. See the note below.

Why every test here scrolls first

Both sections sit below the fold on a normal viewport, and `scrollIntoViewIfNeeded()` appears before each assertion. It is worth being precise about what that call is and is not for.

Playwright's `toBeVisible()` does **not** require an element to be inside the viewport — it checks that the element is attached and has a non-empty bounding box. So the scroll is not there to satisfy the matcher. It is there because the app renders parts of this page lazily: content below the fold can be absent from the DOM until something scrolls toward it. Scrolling first turns "not rendered yet" into "rendered", which is the state the test actually means to check.

Each scroll has its errors swallowed with `.catch(() ⇒ {})`. If the element is already in view, or the page is short enough that there is nothing to scroll, that is not a failure — and letting the scroll throw would replace a clear "Description not visible" with a confusing scroll error.

> **Test 18 reads the href and then does nothing with it**
> The last line prints `href`, which is the one piece of information worth checking — a Creative Commons link that points somewhere else is a licensing problem, not a cosmetic one. Adding `expect(href).toContain('creativecommons.org')` after the skip guard would cost nothing and would turn a printed string into a checked fact. As written, the test proves only that a link matching one of two patterns exists.

9 Group 5 — Navigation

Three tests, lines 465–524. The three ways off this page — to the organisation, backwards, and forwards into the player. All three are behind a skip guard, and they are last in the file because each one navigates away.

#	Test · line	Locator and flow	What it proves

19	**Author opens the org page**
	line 465
		`locator('.details .blue-link, .blue-link.cursor-pointer')`
		`.or(locator('.blue-link, [class*="blue-link"]')`
		`  .filter({ hasText: /alliance|council|foundation|jhpiego|unfpa|others|india/i`
		`})).first()`
		
		`skipIfAbsent(…, 'Author org link')`
		`Promise.all([ waitForEvent('popup', 2500), author.click() ])`
		`target = popup ?? page`
		`expect(seen).toBeGreaterThan(0)`
			conditional That the organisation name is clickable and lands on an org-style page. Counts how many of five markers it can see — *courses offered*, *filter by language*, *english*, *हिंदी*, *courses* — and requires at least one. **It deliberately does not check the course count**, which varies per organisation and would make the test a data check.

20	**Back icon leaves the overview**
	line 497
		`locator('ws-widget-btn-page-back button, ws-widget-btn-page-back')`
		`.or(getByRole('button').filter({ hasText: /chevron_left|arrow_back/i }))`
		`.or(locator('mat-icon:has-text("chevron_left")')`
		`  .locator('xpath=ancestor-or-self::button[1]')).first()`
		
		`click · waitForURL(not a TOC overview, 10 s)`
		`expect(page.url()).not.toMatch(/\/toc\/.*\/overview/)`
			conditional That back actually navigates. The assertion is on the **URL, not on a visible element** — the strongest available statement, because a back button that repaints the same page without changing the route is exactly the bug this catches. The source comment says it plainly: "this one IS asserted".

21	**Start/Resume opens the player**
	line 516
		`const urlBefore = page.url()`
		`const started = await startOrResume(page)`
		`skipIfAbsent(started, 'Start/Resume button')`
		`console.log(page.url() ≠ urlBefore, page.url())`
			observe That the six-locator click found something and the URL moved. There is **no `expect()`** — see the note below.

Three locator techniques worth stealing

1. **Register the listener before the click, in one call.** Test 19 uses `Promise.all([ waitForEvent('popup'), author.click() ])`. The popup promise is created first, so the listener is attached before the click can fire — a tab that opens instantly is not missed. Awaiting the click first and then waiting for the popup is the classic race, and it fails intermittently rather than always.
2. **Climb from the icon to its button.** Material icons render their name as ligature text inside a `<mat-icon>`, so the thing you can find by text is not the thing you can click. Test 20's third fallback, `locator('xpath=ancestor-or-self::button[1]')`, walks up from the matched icon to the nearest enclosing button — and `ancestor-or-self` rather than `ancestor` covers the build where the icon *is* the button.
3. **Handle both tab shapes with one line.** `const target = popup ?? page`. If a new tab opened, assert against it; if the app navigated in place, assert against the same page. Everything after that line is identical, so the test does not fork.

> **Test 21 cannot fail — and it is the one that matters most**

> `startOrResume()` returns `true` only if the URL actually changed (or we were already in the player). So `started === false` means the learner could not start the course — the single most serious thing this page can get wrong. The test passes that value to `skipIfAbsent()`, which **skips**. A broken Start button therefore produces a skipped test, not a red one, and a skip in a report of 21 is easy to read past.
>
> The fix is one line: replace the guard with `expect(started).toBe(true)`. The skip idiom is right for the Creative Commons link, which genuinely may not exist on a given course; it is wrong here, because *every* course has a way in. That change would take the file from 15 unconditional assertions to 16, and would make the suite fail loudly on the defect a learner would notice first.

Tests 19 and 20 use the same guard more defensibly. A course with no linked organisation is a real content state, and some builds of the shell render no back control at all — skipping on those is correct. But both of those tests then go on to assert properly, which is the difference: their guard decides *whether* to check, not *what* to conclude.

10 What green actually proves

Twenty-one green ticks are not twenty-one guarantees. Here is exactly what each one is worth.

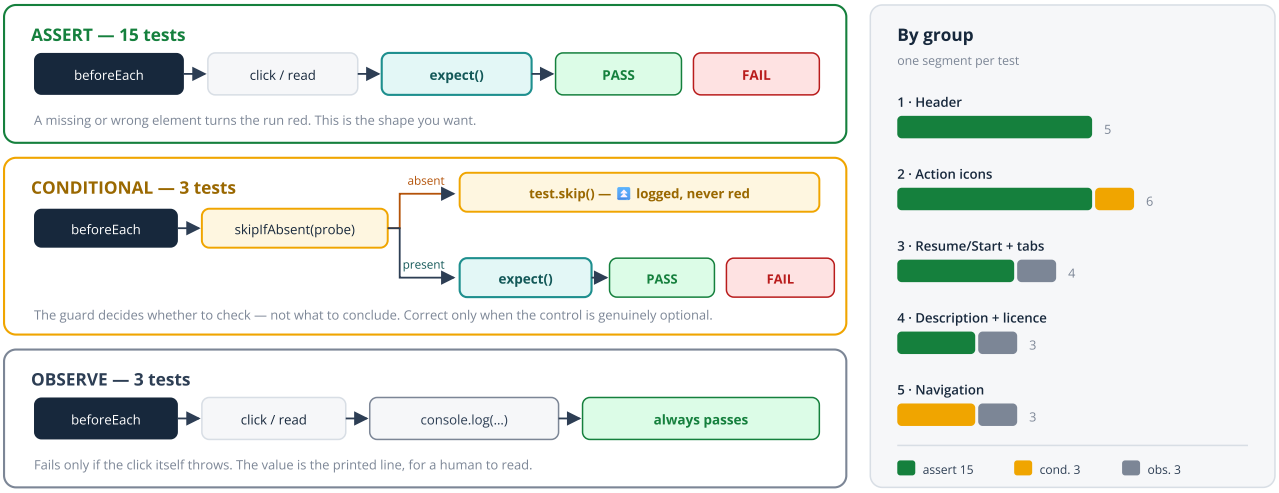


Figure 3 — the three test shapes in this file, and how the 21 tests divide between them. Only the grey segments can never turn red.

The honest summary

Green means this, for certain

- The course opened from a search by name
- Title, CNE hours, duration, review count and author all rendered
- All three action icons are present and clickable
- Details opens **and closes**
- Certificate responds with something
- Share reveals share options
- The banner rendered within 90 seconds
- All three tabs are present
- **The module list is non-empty**
- Description and licence blocks rendered

Green does *not* mean this

- That the star rating rendered — only the `(n)` count is checked
- That the Discussion tab shows anything
- That the Creative Commons link points at creativecommons.org
- **That a learner can start the course** — test 21 skips instead of failing
- That the author link or back icon exist — both skip when absent
- That the banner was *fast*. Slowness is logged, never failed
- That any of the popups contain correct content — only that they appear

Where this file sits among the suites

Across the four scripts documented in this series, the proportion of tests that assert unconditionally varies more than you would expect:

Suite	Tests	Assert always	Shape of the rest
Course TOC this one	21	**15 · 71%**	3 conditional, 3 observe. With the conditionals counted, **86%** of the file can fail.
Public Page	64	42 · 66%	22 soft or present-only.
Public Login	29	18 · 62%	7 present-only, 4 log-only. No test asserts that login succeeded.

Suite	Tests	Assert always	Shape of the rest
My Courses	49	17 · 35%	1 conditional, 31 observe — content assertions depend on what the account happens to have studied.

Why this file scores so much better than My Courses

It is not that this suite is written more carefully. It is that **the TOC overview has data guarantees the My Courses page does not.** Every published course has a title, a duration, an author, a description and at least one module — so those can be asserted outright. My Courses shows whatever the logged-in account has studied, so the same style of assertion would fail on a fresh user. The lesson generalises: the strength of a suite is capped by how much the page is guaranteed to contain.

11 Artifacts and run variants

Every run of this suite produces four reports and a screenshot of all 21 tests, whether they passed or not. The console log is where the findings this suite is actually good at end up.

What a run leaves behind

Artifact	Where it lands, and when
Branded HTML report	`brand-reports/` — written on every run by `reporters/brand-html-reporter.ts`.
Full run report	`test-reports/run-reports/` — HTML *and* PDF, carrying every step and every console line. This is the one to open when you want the 🐢 and ⏭️ lines in context.
Excel summary	Written by `reporters/excel-reporter.ts` on every run.
Email	A no-op unless `SMTP_HOST` and `ALERT_EMAIL_TO` are both set.
Screenshots	`screenshot: 'on'` — captured for **every** test, not only failures, so the branded report shows visual evidence for a green run too.
Video	`retain-on-failure`. A failed popup test gives you the click and the non-response.
Trace	`on-first-retry`. Open with `npx playwright show-trace` for a step-by-step DOM timeline.

The console lines worth grepping for

Six emoji markers carry the information that does not appear anywhere in the pass/fail counts:

Mark	Line	What it tells you
🐢	SLOW: the banner took 41.2s to render	A performance finding from a **passing** test. Printed by `beforeEach` and again by test 12. Count these across runs to see the TOC getting slower before a user complains.
⚠️	Banner (Start/Resume) never rendered	From `beforeEach`. It does not throw — it warns, and lets test 12's assertion produce the real failure with a proper message.
⏭️	<control> not present on this course — skipping	Which optional controls this course lacks. Three tests can print this; if test 21 does, read the red callout in Section 9.
↻	TOC not loaded from cached URL · open attempt N failed	The fast path missed, or a search attempt flaked. One or two of these in a run is normal; one before every test means the cache is never valid and the run is paying the slow path 21 times.
📌	Course TOC URL: .../toc/do_.../overview	The resolved course id. Paste it into a browser to look at exactly what the suite was looking at.
✅	Modules tab — 14 module/resource item(s)	The module count. The assertion only requires more than zero, so a drop from 14 to 1 passes — this line is the only place that change is visible.

```
# pull the performance findings out of a run
npm run run:course-toc 2>&1 | Select-String "🐢|⚠️|⏭️"
```

Run variants

Command	Use it when
`npm run run:course-toc`	Normal use. Headed, against staging. Needs `COURSE_NAME`.
`HEADLESS=true npm run run:course-toc`	Unattended. Roughly twice as fast, and the screenshots are still captured.
`… --grep "popup"`	The four popup tests only — the quickest loop when fixing a dialog selector.
`… --repeat-each=5`	Hunting flake in the banner timing. Watch the 🐢 lines rather than the pass count.
`USE_CDP=1 …`	Local development against localhost, where cookie injection returns 419 and only attaching to a live Chrome works.
`TOC_BANNER_TIMEOUT=180000 …`	A CI box slow enough that 90 seconds is not enough. All four knobs from Section 3 read the environment.

In `report:app`, this suite currently contributes nothing

`npm run report:app` runs Account, My Courses, **Course TOC** and Competency into one blob report — but it does not set `COURSE_NAME`, and `.env` does not define it either. So in that run all 21 tests skip, and the app report has a Course TOC section with no results in it. The fix is one word in `package.json`: add `COURSE_NAME="<a course that exists on the target env>"` to the `report:app` script, exactly as `report:course` already does for the completion suite.

12 Failure modes and fixes

What each shape of failure means, in the order you are most likely to meet it.

Symptom	Cause	Fix
All 21 tests skipped, none red	`COURSE_NAME` is not set. `beforeEach` skips at line 223 before doing anything else.	Set it. A configuration state, not a defect — but it reads as a clean run in a summary count, so check the skip total before trusting green.
`Error: Could not open course "…"` on every test	Three search attempts all failed (line 250). Either the name does not match anything on this environment, or the home search box never rendered.	Check the exact title on the target environment: the match is on the **first 8 characters**, so a changed prefix breaks it. Confirm `BASE_URL`.
`↻ TOC not loaded from cached URL` before *every* test	`onToc()` keeps returning false. Usually the session expired and the app is redirecting to login, so the URL never matches `/toc/…/overview`.	Re-authenticate. In CDP mode log the attached Chrome back in; otherwise delete the stale `storageState` file so the harness logs in again.
`⚠️ Banner never rendered`, then test 12 fails	The enrolment or progress call is failing or timing out. The banner is the only element that depends on it.	Watch the network panel on the URL from the 📌 line. Slow-but-successful calls mean raising `TOC_BANNER_TIMEOUT`; failing ones mean the test is right.
Banner test passes but prints `🚗` on every run	Real slowness. Most often seen on the run right after a course has been opened and exited.	Not a test fix. Report the number — that is what the line exists for.
Test 8 or 10 fails: the popup **opened** but would not close	`closeAnyPopup()`'s five close-button shapes no longer match, and Escape did not dismiss the dialog either.	Add the dialog's close-control shape to the selector at line 161. Fix it properly rather than reaching for Escape — a dialog a user cannot close is the bug this test exists to find.
An action-icon click appears to do nothing	The click landed on the label span instead of the `div.toc-icon-action` wrapper that carries the handler.	Use `actionIcon()`. Do not "simplify" it to `getByText('Details').click()` — that is the exact regression it was written to prevent.
The banner locator finds nothing, **only on one account**	That account completed the course, so the control is now an `<a class="banner-btn resumeButton">Start Again</a>` and neither role matches.	Keep `.banner-btn` first in the locator chain. See Section 3.
Test 14 fails: `expected > 0, received 0`	The module list is genuinely empty — the content hierarchy did not load.	A real defect, and the most valuable failure this suite produces. Confirm by opening the 📌 URL and clicking Modules by hand.
`strict mode violation: resolved to N elements`	A `.first()` was dropped from a locator during an edit.	Put it back. Every locator in this file ends in `.first()` for exactly this reason.
Passes headed locally, fails in CI	CDP mode has no live Chrome to attach to on a build agent.	CI should use mode 4 — `TEST_USER_EMAIL` and `TEST_USER_PASSWORD`. Leave `USE_CDP` unset there.
Tests 19 and 20 skip on every course	Either the courses genuinely have no linked organisation and no back control, or `.blue-link` / `ws-widget-btn-page-back` were renamed in the shell.	Check one course by hand. If the controls are visibly there, the selectors are stale — and a permanently-skipping test is worse than a failing one, because nothing draws attention to it.

The one change worth making first

If you take a single action from this document, make it the one-line fix in Section 9: swap `skipIfAbsent(started, 'Start/Resume button')` for `expect(started).toBe(true)` at line 522. It converts the most serious failure this page can have — a learner unable to start the course — from a quiet skip into a red test.